HW 4.3 Reproducible Data Analysis

Jiawei Xi

2026-04-13

1 Busiest Airports Analysis

This section examines passenger traffic trends for six major airports from 2020 through 2025. Table Table 1 summarizes the annual passenger totals for the selected airports, while Figure Figure 1 shows how passenger traffic changed over time. Together, these visualizations show both the overall scale of passenger traffic and the differences in recovery patterns across airports.

The table is useful for comparing exact passenger totals by airport and year. In contrast, the line graph makes it easier to see changes over time, especially the declines and rebounds that occurred across the 2020 to 2025 period. Looking at both visualizations together helps show that some airports recovered more steadily, while others experienced sharper increases after the lowest-traffic years. This combination of a table and a plot provides a clearer understanding of the busiest-airports context than either display would on its own.

Table 1: Total annual passengers from 2020 to 2025 for six busy airports.

Airport	IATA	2020	2021	2022	2023	2024
Hartsfield–Jackson Atlanta International Airport	ATL	42,918,685	75,704,760	93,699,630	104,653,451	108,067,766
O'Hare International Airport	ORD	30,860,251	54,020,399	68,340,619	73,894,226	80,043,050
Heathrow Airport	LHR	22,111,469		61,614,508	79,183,364	83,884,572
Phoenix Sky Harbor International Airport	PHX	21,928,708	38,846,713	44,397,854	48,654,432	52,325,266
Frankfurt Airport	FRA	18,768,601	24,812,849	48,918,482	59,355,389	61,564,957
Beijing Daxing International Airport	PKX		25,051,012			49,441,029

"'qmd # Monte Carlo Numerical Integration

This section uses Monte Carlo simulation to estimate the value of the definite integral of the standard normal density over the interval from -4 to 4. The method works by randomly generating points inside a rectangular window, identifying which points fall on or below the

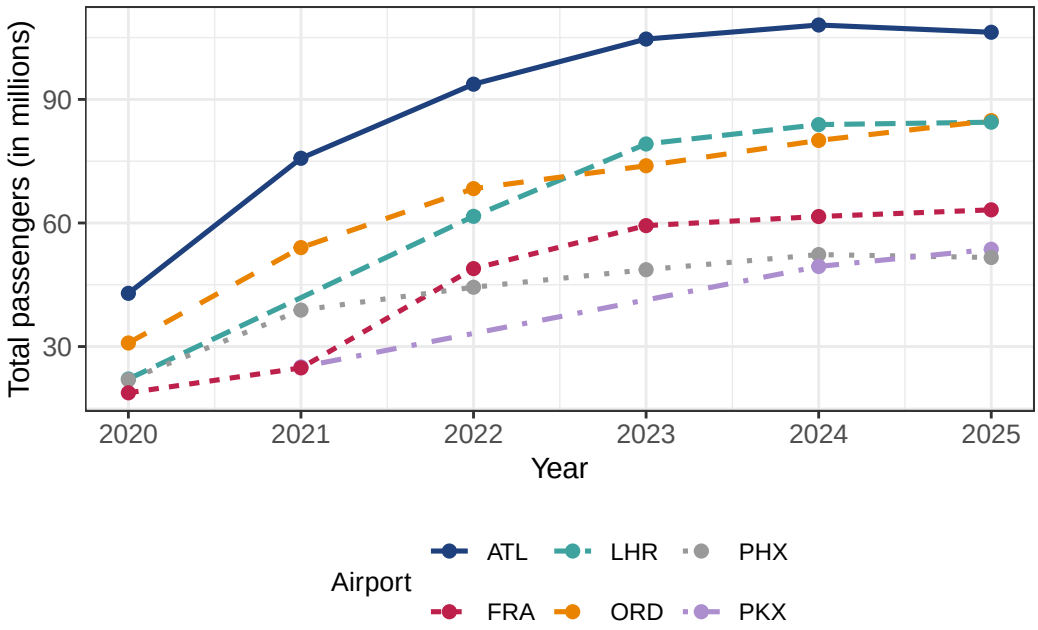


Figure 1: Passenger traffic trends from 2020 to 2025 for six busy airports.

density curve, and then multiplying that proportion by the area of the rectangle. In this case, the rectangular window has width 8 and height 0.4, so its area is 3.2.

Figure Figure 2 shows the simulation at four different resolutions. When the number of simulated points is small, the estimated value varies more because random fluctuation has a stronger effect on the proportion of points below the curve. As the number of points increases, the point cloud fills the rectangle more evenly and the estimated integral becomes more stable. This visual pattern suggests that the Monte Carlo estimate improves as the sample size increases.

Based on the higher-resolution panels, the value of the integral appears to be approximately 1. This conclusion is reasonable because the standard normal density has almost all of its probability mass between -4 and 4, and the estimates in the larger simulations move closer to 1 than the estimates in the lower-resolution panels. The two largest panels provide the strongest support for this conclusion because they show less variability and better approximate the area under the curve.

Planning and Prompting: Generic Response

To explore how generative AI can support data analysis, I used a generic prompt that asks for a broad explanation of a typical data analysis workflow. The prompt I used was:

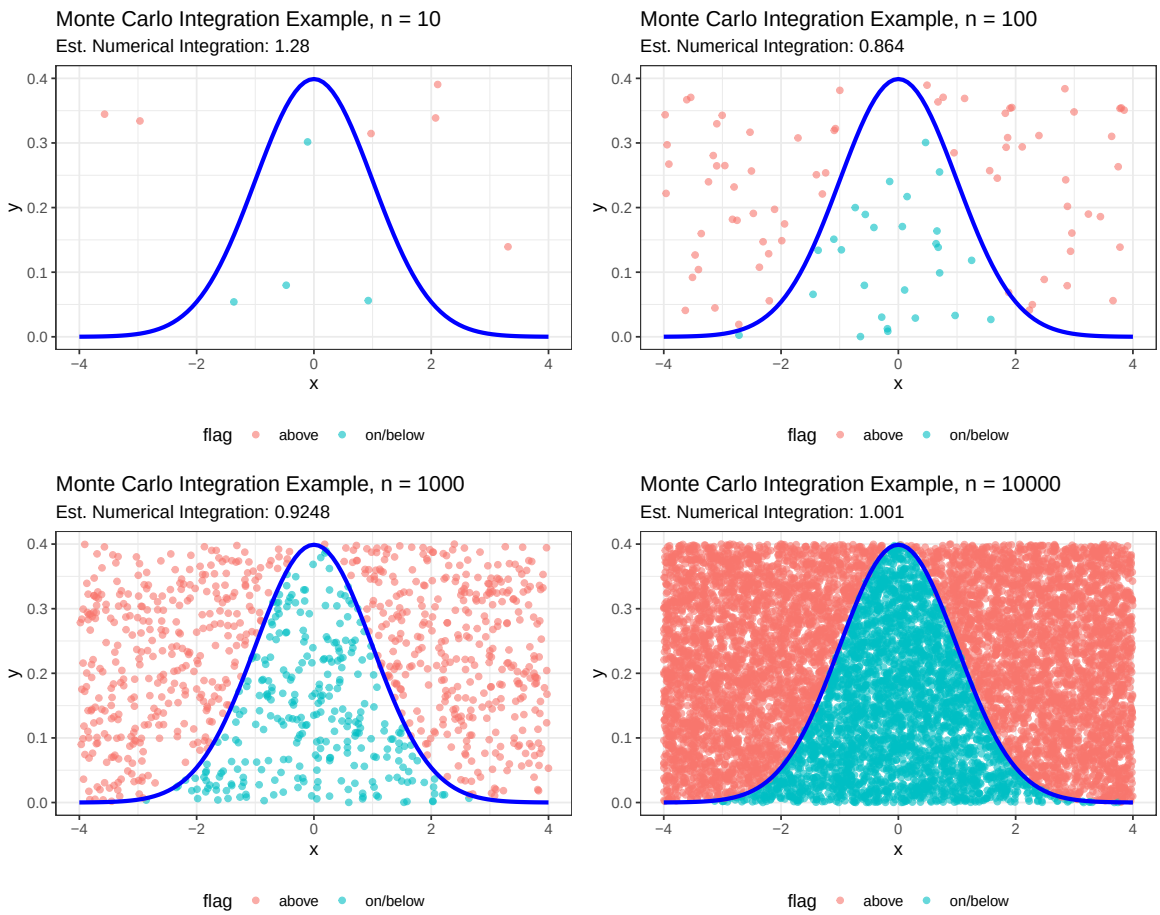
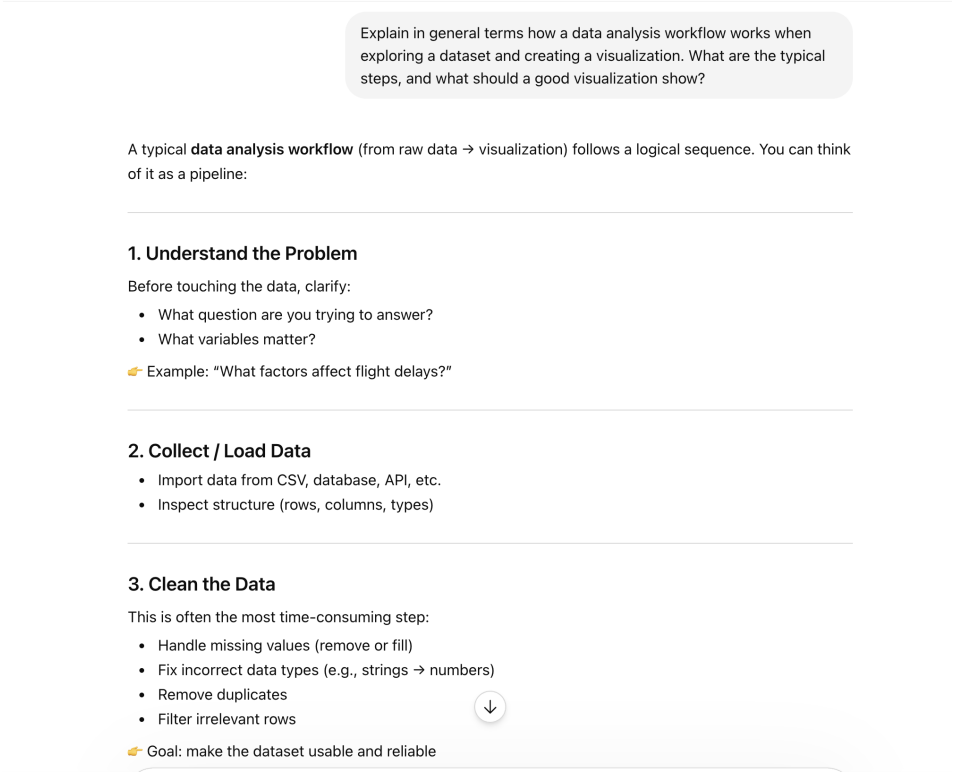


Figure 2: Monte Carlo numerical integration for the standard normal density using four simulation resolutions.

"Explain in general terms how a data analysis workflow works when exploring a dataset and creating a visualization. What are the typical steps, and what should a good visualization show?"

Figure Figure 3 shows the response generated by the AI tool.



Explain in general terms how a data analysis workflow works when exploring a dataset and creating a visualization. What are the typical steps, and what should a good visualization show?

A typical **data analysis workflow** (from raw data → visualization) follows a logical sequence. You can think of it as a pipeline:

---

1. Understand the Problem

Before touching the data, clarify:

- What question are you trying to answer?
- What variables matter?

👉 Example: "What factors affect flight delays?"

---

2. Collect / Load Data

- Import data from CSV, database, API, etc.
- Inspect structure (rows, columns, types)

---

3. Clean the Data

This is often the most time-consuming step:

- Handle missing values (remove or fill)
- Fix incorrect data types (e.g., strings → numbers)
- Remove duplicates
- Filter irrelevant rows

👉 Goal: make the dataset usable and reliable

Figure 3: Response generated by a generative AI tool using a generic prompt about data analysis workflows.

The AI-generated response describes a general workflow for data analysis, including steps such as understanding the dataset, cleaning and organizing the data, and creating visualizations to communicate patterns. It emphasizes that good visualizations should clearly present trends, comparisons, and key insights in a way that is easy for the audience to understand.

While this response provides a useful high-level overview, it remains broad and does not address the specific details of the dataset or analysis in this assignment. Compared to more targeted prompts, the generic response lacks concrete guidance on implementation decisions. However, it is still valuable for building a general understanding of how data analysis and visualization should be approached.

1.1 Planning and Prompting: Narrative Text

The plan-informed generative AI response provides more targeted and context-specific guidance compared to the generic response. While the generic prompt explains a broad data analysis workflow, it does not incorporate the specific structure, variables, or goals of this assignment. In contrast, the plan-informed response aligns more closely with the dataset and analysis task, offering clearer direction on how to approach the wrangling and visualization steps.

From the visualizations created in this assignment, it is evident that a structured and context-aware approach leads to more meaningful results. For example, the Monte Carlo simulation plots clearly demonstrate how increasing the number of sampled points improves the stability and accuracy of the estimated integral. This reflects a deliberate implementation choice that would not be directly suggested by a generic response alone.

Comparing these approaches highlights that generic prompts are useful for building foundational understanding, but they lack the specificity needed for effective implementation. Plan-informed prompts, on the other hand, bridge this gap by connecting general principles to the actual problem context. My own analysis builds on both: I used the general workflow as a starting point, but relied on a more tailored approach to correctly implement the simulation and produce interpretable visualizations.

Overall, this exercise demonstrates that generative AI is most effective when prompts are specific and aligned with the task. Generic prompts provide useful high-level guidance, but deeper insights and accurate results require incorporating problem-specific details and critical thinking.

1.2 Self-Reflection

Throughout this assignment, I developed a stronger understanding of how data analysis workflows are structured and implemented in practice. One key takeaway is the importance of moving from a general idea to a concrete implementation. For example, in the Monte Carlo simulation section, I learned how increasing the number of sampled points leads to more stable and accurate estimates. This helped me better understand the relationship between randomness, sample size, and convergence.

Another important lesson was the role of data wrangling in preparing data for analysis. Before creating visualizations, I had to carefully organize the data and define variables such as whether points fall above or below the curve. This step was essential for ensuring that the results were meaningful and interpretable.

I also learned how visualization design affects interpretation. By creating multiple plots at different resolutions, I was able to clearly see how patterns become more stable as the sample

size increases. This reinforced the idea that visualizations are not just for presentation, but also for understanding the underlying process.

Overall, this assignment helped me connect theoretical concepts with practical implementation, and improved my ability to think critically about both data analysis and visualization.

1.3 GenAI Usage Appendix

1.3.1 Usage 1: Generic Prompt Response

- **Tool Used:** ChatGPT (OpenAI GPT-5.3)
- **Date:** 2026-04-13
- **Prompt Used:** "Explain in general terms how a data analysis workflow works when exploring a dataset and creating a visualization. What are the typical steps, and what should a good visualization show?"
- **Response:** A typical data analysis workflow (from raw data → visualization) follows a logical sequence:

1. Understand the problem
   - Identify the question being asked
   - Determine relevant variables
2. Collect / load data
   - Import data from files, databases, or APIs
   - Inspect structure (rows, columns, types)
3. Clean the data
   - Handle missing values
   - Correct data types
   - Remove duplicates
   - Filter irrelevant observations
4. Explore and analyze
   - Summarize statistics
   - Identify patterns and relationships
5. Visualize results
   - Use plots to communicate trends and comparisons
   - Ensure clarity, labeling, and readability

Good visualizations should clearly communicate patterns, comparisons, and key insights in a way that is easy for the audience to understand.

1.4 Code Appendix

2 Monte Carlo Integration Code

library(tidyverse) library(patchwork)

3 Function to generate random points

mc_points <- function(n, x_min, x_max, y_min, y_max) { x <- runif(n, min = x_min, max = x_max) y <- runif(n, min = y_min, max = y_max) data.frame(x = x, y = y) }

4 Function to create Monte Carlo plot

make_mc_plot <- function(n) { set.seed(184)

mc_df <- mc_points(n, -4, 4, 0, 0.4) |> mutate(density_y = dnorm(x, mean = 0, sd = 1), flag = case_when(y > density_y ~ "above", TRUE ~ "on/below"), hit = case_when(flag == "on/below" ~ 1, TRUE ~ 0))

est_int <- mean(mc_df$hit) * (4 - (-4)) * (0.4 - 0)

ggplot(mc_df, aes(x = x, y = y, color = flag)) + geom_point(alpha = 0.6, size = 1.8) + stat_function(fun = dnorm, args = list(mean = 0, sd = 1), xlim = c(-4, 4), linewidth = 1.2, color = "blue") + scale_color_manual(values = c("above" = "#F8766D", "on/below" = "#00BFC4")) + labs(title = paste("Monte Carlo Integration Example, n =", n), subtitle = paste("Est. Numerical Integration:", round(est_int, 4)), x = "x", y = "y", color = "flag") + theme_bw() }

5 Generate plots

plot10 <- make_mc_plot(10) plot100 <- make_mc_plot(100) plot1000 <- make_mc_plot(1000) plot10000 <- make_mc_plot(10000)

6 Combine plots

(plot10 + plot100) / (plot1000 + plot10000)